JavaScript Mastery MCQ Quiz

Description: A comprehensive test covering core JavaScript concepts, including syntax, ES6+ features, asynchronous programming, the event loop, and advanced memory management.

Easy Questions

1. What will the expression `typeof NaN` return in JavaScript?
   A. "number"
   B. "NaN"
   C. "undefined"
   D. "object"
   Difficulty: Easy
   Answer: A. "number"
   Explanation: Despite standing for "Not-A-Number", NaN is technically classified as a numeric data type in JavaScript's implementation of the IEEE 754 standard.
2. Which keyword is used to declare a variable that cannot be reassigned?
   A. let
   B. var
   C. const
   D. static
   Difficulty: Easy
   Answer: C. const
   Explanation: The `const` keyword declares a block-scoped local variable that cannot be reassigned to a new value through assignment, nor can it be redeclared.
3. What is the primary difference between `==` and `===` operators?
   A. `==` checks for value equality, while `===` checks for both value and type equality (strict equality).
   B. `==` checks for type, while `===` checks for value.
   C. `===` is only used for strings.
   D. There is no difference; they are interchangeable.
   Difficulty: Easy
   Answer: A. `==` checks for value equality, while `===` checks for both value and type equality (strict equality).
   Explanation: `==` performs type coercion before comparison (e.g., `"1" == 1` is true). `===` does not perform coercion, making it safer to use (e.g., `"1" === 1` is false).
4. Which array method is used to add one or more elements to the end of an array?
   A. unshift()
   B. pop()
   C. push()
   D. concat()
   Difficulty: Easy
   Answer: C. push()
   Explanation: The `push()` method adds new elements to the end of an array and returns the new length of the array. `unshift()` adds to the beginning.

5. What is the default value of an uninitialized variable declared with `let`?
   A. null
   B. undefined
   C. 0
   D. false
   Difficulty: Easy
   Answer: B. undefined
   Explanation: In JavaScript, if a variable is declared but not assigned a value, the engine automatically assigns it the value `undefined`.
6. Which built-in method is used to convert a JSON string into a JavaScript object?
   A. JSON.stringify()
   B. JSON.parse()
   C. JSON.toObject()
   D. String.toJSON()
   Difficulty: Easy
   Answer: B. JSON.parse()
   Explanation: `JSON.parse()` takes a valid JSON string and transforms it into a native JavaScript object. `JSON.stringify()` does the exact opposite.
7. What will `console.log(typeof null)` output?
   A. "null"
   B. "undefined"
   C. "object"
   D. "boolean"
   Difficulty: Easy
   Answer: C. "object"
   Explanation: This is a well-known, historical bug in JavaScript. `null` is a primitive value, but `typeof null` incorrectly returns "object". It cannot be fixed without breaking legacy code.
8. What syntax is used to create a template literal (template string) in modern JavaScript?
   A. Single quotes (' ')
   B. Double quotes (" ")
   C. Backticks (` `)
   D. Brackets ([ ])
   Difficulty: Easy
   Answer: C. Backticks (` `)
   Explanation: Template literals are enclosed by backtick characters and allow for multi-line strings and string interpolation using the `${expression}` syntax.
9. Which DOM method is best for selecting a single HTML element based on its unique ID attribute?
   A. document.querySelector()
   B. document.getElementsByClassName()
   C. document.getElementById()
   D. document.querySelectorAll()
   Difficulty: Easy

Answer: C. document.getElementById()
Explanation: getElementById is the fastest and most direct native method to select a specific, unique element from the DOM using its id attribute.

Medium Questions

11. What is the fundamental difference between the Array methods map() and forEach()?
    A. map() alters the original array, forEach() does not.
    B. forEach() returns a new array, map() returns undefined.
    C. map() returns a new array containing the results of the callback, while forEach() returns undefined.
    D. map() can only be used on numbers.
    Difficulty: Medium
    Answer: C. map() returns a new array containing the results of the callback, while forEach() returns undefined.
    Explanation: map() is used to transform data into a new array. forEach() is simply used to execute a side-effect (like logging or mutating an external variable) for every item.
12. Which of the following best defines a "Closure" in JavaScript?
    A. A function that takes another function as an argument.
    B. A function bundled together with its lexical environment, allowing it to remember the variables from its outer scope even after the outer function has executed.
    C. The process of removing unused variables from memory.
    D. A block of code that handles asynchronous errors.
    Difficulty: Medium
    Answer: B. A function bundled together with its lexical environment, allowing it to remember the variables from its outer scope even after the outer function has executed.
    Explanation: Closures are created every time a function is created. They allow inner functions to access the scope of their enclosing functions, enabling data privacy and factory patterns.
13. What are the three possible states of a Promise?
    A. Waiting, Running, Finished
    B. Pending, Fulfilled, Rejected
    C. Start, Success, Error
    D. Resolved, Unresolved, Cancelled
    Difficulty: Medium
    Answer: B. Pending, Fulfilled, Rejected
    Explanation: A Promise starts in the pending state. If the operation is successful, it becomes fulfilled. If it fails, it becomes rejected.
14. How do Arrow Functions (() => {}) differ from traditional function expressions regarding the this keyword?
    A. Arrow functions do not have their own this; they inherit this from the surrounding lexical scope.
    B. Arrow functions permanently bind this to the global window object.
    C. Arrow functions strictly bind this to the object that called them.

D. There is no difference.
Difficulty: Medium
Answer: A. Arrow functions do not have their own `this`; they inherit `this` from the surrounding lexical scope.
Explanation: Traditional functions dynamically bind `this` based on how they are called. Arrow functions resolve `this` lexically, making them ideal for callbacks inside classes or objects.

15. What is the concept of "Hoisting" in JavaScript?
    A. The automatic conversion of primitive types to objects.
    B. JavaScript's default behavior of moving variable and function declarations to the top of their respective scopes during the compile phase.
    C. Compressing JavaScript files for production.
    D. Elevating local variables to global scope.
    Difficulty: Medium
    Answer: B. JavaScript's default behavior of moving variable and function declarations to the top of their respective scopes during the compile phase.
    Explanation: Hoisting allows functions to be safely used in code before they are declared. Note that `var` is hoisted and initialized to `undefined`, while `let` and `const` are hoisted but uninitialized (Temporal Dead Zone).
16. What is the output of `console.log(1 + "2" + "2")` and `console.log(1 + +"2" + "2")`?
    A. "122" and "32"
    B. "5" and "5"
    C. "122" and "122"
    D. "14" and "32"
    Difficulty: Medium
    Answer: A. "122" and "32"
    Explanation: In the first expression, the `+` acts as a string concatenator because of the string `"2"`, resulting in "122". In the second, `+"2"` is a unary plus operator that converts "2" to a number. So `1 + 2` is `3`, and then `3 + "2"` concatenates to "32".
17. What is the primary purpose of the `reduce()` method on an array?
    A. To filter out unwanted elements based on a condition.
    B. To reverse the order of elements.
    C. To execute a reducer callback function on each element, passing in the return value from the calculation on the preceding element, ultimately resulting in a single output value.
    D. To find the largest number in an array.
    Difficulty: Medium
    Answer: C. To execute a reducer callback function on each element, passing in the return value from the calculation on the preceding element, ultimately resulting in a single output value.
    Explanation: `reduce()` is powerful for summing numbers, flattening arrays, or transforming an array of objects into a single dictionary object.
18. What does "Event Bubbling" mean in the DOM?
    A. An event starts at the deepest target element and triggers handlers on its parents,

moving upwards to the document root.
B. An event starts at the root document and moves downwards to the target element.
C. Multiple events firing at the exact same time.
D. Removing all event listeners to free up memory.
Difficulty: Medium
Answer: A. An event starts at the deepest target element and triggers handlers on its parents, moving upwards to the document root.
Explanation: By default, events bubble up. If you click a `<button>` inside a `<div>`, the button's click handler fires first, followed by the div's click handler, unless `event.stopPropagation()` is called.

19. Using ES6 Destructuring, how would you extract the `name` property from an object `user` and assign it to a new variable called `userName`?
A. const { userName: name } = user;
B. const { name = userName } = user;
C. const { name: userName } = user;
D. const userName = { ...user.name };
Difficulty: Medium
Answer: C. const { name: userName } = user;
Explanation: The syntax `{ originalProperty: newVariableName }` allows you to extract a value from an object and assign it to a variable with a completely different name.
20. What does the Rest operator (`...`) do when used in a function parameter definition like `function sum(...args)`?
A. It pauses the function execution.
B. It collects all remaining arguments passed to the function into a true JavaScript Array.
C. It expands an array into individual arguments.
D. It skips the remaining arguments.
Difficulty: Medium
Answer: B. It collects all remaining arguments passed to the function into a true JavaScript Array.
Explanation: The rest parameter syntax allows a function to accept an indefinite number of arguments as an array. (Note: when `...` is used in a function call, it is the Spread operator, which does the exact opposite).

Advanced Questions

21. In the JavaScript Event Loop, what is the execution priority difference between Macrotasks (like `setTimeout`) and Microtasks (like `Promise.then`)?
A. Macrotasks always execute before Microtasks.
B. They execute randomly based on the browser engine.
C. The Microtask queue is entirely emptied after the current synchronous code finishes, before the Event Loop moves on to process the next Macrotask.
D. They are executed at the exact same time.
Difficulty: Advanced
Answer: C. The Microtask queue is entirely emptied after the current synchronous code

finishes, before the Event Loop moves on to process the next Macrotask.
Explanation: Promises are Microtasks. If a `setTimeout` (0ms delay) and a `Promise.resolve()` are both queued, the Promise callback will always run first because the Event Loop prioritizes the Microtask queue.

22. How does Prototypal Inheritance work in JavaScript?
    A. Classes are strictly copied during instantiation, just like in Java.
    B. Objects contain a hidden `[[Prototype]]` reference to another object. If a property is not found on the object itself, JavaScript traverses up this prototype chain to find it.
    C. Interfaces dictate object structure at runtime.
    D. Objects inherit properties by physically copying them from the parent upon creation.
    Difficulty: Advanced
    Answer: B. Objects contain a hidden `[[Prototype]]` reference to another object. If a property is not found on the object itself, JavaScript traverses up this prototype chain to find it.
    Explanation: JavaScript does not have traditional classical inheritance. Instead, objects link to other objects via the prototype chain. The `class` keyword introduced in ES6 is mostly syntactic sugar over this prototypal system.
23. What is the fundamental difference between the `call()` and `bind()` methods?
    A. `call()` executes the function immediately with a specified `this` value, while `bind()` returns a new function with `this` permanently bound, to be executed later.
    B. `bind()` accepts arguments as an array, `call()` does not.
    C. `call()` only works on DOM elements.
    D. There is no difference; they are aliases.
    Difficulty: Advanced
    Answer: A. `call()` executes the function immediately with a specified `this` value, while `bind()` returns a new function with `this` permanently bound, to be executed later.
    Explanation: Both allow you to explicitly define what `this` refers to. `call` and `apply` are for immediate execution, whereas `bind` is used to create a partially applied function for future execution (like in React event handlers).
24. Which data structures in JavaScript allow the Garbage Collector to remove their elements if there are no other references to them, preventing memory leaks?
    A. Map and Set
    B. Array and Object
    C. WeakMap and WeakSet
    D. String and Symbol
    Difficulty: Advanced
    Answer: C. WeakMap and WeakSet
    Explanation: In a standard `Map`, keys are strongly held; even if the original object is deleted, the Map keeps it in memory. In a `WeakMap`, the keys are "weakly" held, allowing the garbage collector to clear them if no other references exist.
25. What is a Generator function (`function*`) in JavaScript?
    A. A function that automatically generates random numbers.
    B. A function that can pause its own execution mid-way using the `yield` keyword and resume later, maintaining its internal state.

C. A function used exclusively for creating HTML elements.
D. A function that generates unique IDs.
Difficulty: Advanced
Answer: B. A function that can pause its own execution mid-way using the `yield` keyword and resume later, maintaining its internal state.
Explanation: Generators return an Iterator object. When you call `.next()` on the iterator, the function runs until it hits a `yield`, returns the yielded value, and pauses until `.next()` is called again.

26. What does the concept of the "Temporal Dead Zone" (TDZ) refer to?
    A. The time it takes for an API to respond.
    B. The period in a block's execution before a `let` or `const` variable is declared, during which accessing the variable throws a ReferenceError.
    C. A memory leak caused by closures.
    D. The delay inside a `setTimeout`.
    Difficulty: Advanced
    Answer: B. The period in a block's execution before a `let` or `const` variable is declared, during which accessing the variable throws a ReferenceError.
    Explanation: Unlike `var` which is initialized to `undefined` during hoisting, `let` and `const` are hoisted but remain uninitialized. Accessing them in the TDZ guarantees developers catch initialization errors early.
27. What is the primary use case for JavaScript Proxies?
    A. To bypass CORS restrictions in the browser.
    B. To intercept and define custom behavior for fundamental object operations (like property lookup, assignment, enumeration, and function invocation).
    C. To compress HTTP requests.
    D. To securely store passwords.
    Difficulty: Advanced
    Answer: B. To intercept and define custom behavior for fundamental object operations (like property lookup, assignment, enumeration, and function invocation).
    Explanation: A Proxy allows you to create a "trap" for an object. Vue 3 and MobX use Proxies extensively to make objects reactive by intercepting `get` and `set` calls automatically.
28. In the module system (ESM), what is the difference between a default export and a named export?
    A. Named exports require curly braces `{}` when imported and must use the exact name, while a default export can be imported with any name without braces.
    B. Default exports are hoisted, named exports are not.
    C. Named exports can only export strings.
    D. Default exports can only be used in Node.js.
    Difficulty: Advanced
    Answer: A. Named exports require curly braces `{}` when imported and must use the exact name, while a default export can be imported with any name without braces.
    Explanation: A file can have multiple named exports (`export const a = 1`), imported like

import { a } from './file'. A file can only have one default export (export default MyClass), imported like import CustomName from './file'.

29. What is a "Curried" function in functional programming?
    A. A function that handles DOM events.
    B. A function that transforms multiple arguments into a sequence of nested functions, each taking a single argument (e.g., f(a, b, c) becomes f(a)(b)(c)).
    C. A function optimized by the V8 engine.
    D. A function that automatically deeply clones an object.
    Difficulty: Advanced
    Answer: B. A function that transforms multiple arguments into a sequence of nested functions, each taking a single argument (e.g., f(a, b, c) becomes f(a)(b)(c)).
    Explanation: Currying allows you to partially apply arguments to a function, creating specialized functions from generic ones. It is heavily used in libraries like Ramda and Redux.
30. How does JavaScript handle integers versus floating-point numbers internally?
    A. It uses separate 32-bit spaces for integers and floats.
    B. It treats everything as a string internally.
    C. JavaScript does not differentiate; all numbers are strictly 64-bit double-precision floating-point numbers (IEEE 754).
    D. It relies on the operating system's architecture.
    Difficulty: Advanced
    Answer: C. JavaScript does not differentiate; all numbers are strictly 64-bit double-precision floating-point numbers (IEEE 754).
    Explanation: Because all numbers are floats, JavaScript struggles with precise math at extreme limits. This is why 0.1 + 0.2 === 0.3 evaluates to false (it evaluates to 0.30000000000000004), and why BigInt was introduced for massive integers.

Easy Questions

31. What will typeof [] return in JavaScript?
    A. "array"
    B. "object"
    C. "list"
    D. "undefined"
    Difficulty: Easy
    Answer: B. "object"
    Explanation: In JavaScript, arrays are a special type of object. To specifically check if a variable is an array, you should use Array.isArray().
32. What is the difference between declaring a variable with var versus let?
    A. var is block-scoped, while let is function-scoped.
    B. var variables cannot be reassigned.
    C. var is function-scoped (or globally scoped), while let is strictly block-scoped.
    D. There is no difference in modern JavaScript.

Difficulty: Easy
Answer: C. var is function-scoped (or globally scoped), while let is strictly block-scoped.
Explanation: Variables declared with let inside a { } block cannot be accessed from outside that block, preventing accidental variable overrides. var ignores block scope unless it's a function.

33. How do you find the number of characters in a string named myString?
    A. myString.size()
    B. myString.length
    C. myString.count
    D. myString.length()
    Difficulty: Easy
    Answer: B. myString.length
    Explanation: In JavaScript, length is a property of the String object (not a method/function), so it is accessed without parentheses.
34. What is the purpose of the setTimeout function?
    A. To pause the entire JavaScript engine for a specified time.
    B. To execute a function repeatedly at a set interval.
    C. To execute a function once after a specified delay in milliseconds.
    D. To measure how long a function takes to run.
    Difficulty: Easy
    Answer: C. To execute a function once after a specified delay in milliseconds.
    Explanation: setTimeout schedules a callback to run after a minimum delay. setInterval is used to run it repeatedly.
35. Which built-in function is used to check if a value is technically "Not-A-Number"?
    A. isNaN()
    B. isNotNumber()
    C. Number.isInvalid()
    D. typeof value === "NaN"
    Difficulty: Easy
    Answer: A. isNaN()
    Explanation: The global isNaN() function determines whether a value is NaN or not. (Note: Number.isNaN() is a stricter, more modern version introduced in ES6).
36. What happens if you manually set the length property of an array to 0?
    A. It throws an error.
    B. The array is instantly emptied of all its elements.
    C. The length property updates, but the elements remain.
    D. Only the first element is deleted.
    Difficulty: Easy
    Answer: B. The array is instantly emptied of all its elements.
    Explanation: The length property of an array is writable. Setting it to 0 is a fast and common way to completely clear an existing array in JavaScript.
37. What does Math.random() return?
    A. A random integer between 1 and 100.
    B. A random floating-point number between 0 (inclusive) and 1 (exclusive).

C. A random boolean value.
D. A random character from the alphabet.
Difficulty: Easy
Answer: B. A random floating-point number between 0 (inclusive) and 1 (exclusive).
Explanation: `Math.random()` generates a decimal like 0.4582. To get a whole number, you must multiply it by your desired range and use `Math.floor()` or `Math.ceil()`.

38. Which console method is specifically used to output an error message to the web console, often styled in red?
    A. console.log()
    B. console.warn()
    C. console.error()
    D. console.debug()
    Difficulty: Easy
    Answer: C. console.error()
    Explanation: `console.error()` prints the message with a red icon and text, and often captures a stack trace, making it much easier to spot errors in the developer tools.
39. Which string method checks if a specific sequence of characters exists within another string?
    A. contains()
    B. has()
    C. includes()
    D. match()
    Difficulty: Easy
    Answer: C. includes()
    Explanation: `includes()` returns `true` if the search string is found anywhere within the given string, and `false` if not. It is case-sensitive.
40. What does the `++` operator do in JavaScript?
    A. It concatenates two strings.
    B. It adds 2 to a number.
    C. It increments a numeric variable by 1.
    D. It duplicates a variable.
    Difficulty: Easy
    Answer: C. It increments a numeric variable by 1.
    Explanation: The increment operator `++` adds one to its operand. It can be used as a prefix (`++x`) or postfix (`x++`), which changes when the value is returned during the operation.

Medium Questions

41. What is the difference between the Array methods `find()` and `filter()`?
    A. `find()` returns the first matching element, `filter()` returns a new array of all matching elements.
    B. `find()` searches strings, `filter()` searches objects.
    C. `find()` returns an array, `filter()` returns a boolean.

D. There is no difference; they are aliases.
Difficulty: Medium
Answer: A. find() returns the first matching element, filter() returns a new array of all matching elements.
Explanation: Use find() when you only need a single specific item (it stops iterating once found). Use filter() when you need all items that meet a certain condition.

42. How can you extract all the keys from a JavaScript object into an array?
    A. Object.values(obj)
    B. obj.getKeys()
    C. Object.keys(obj)
    D. Array.from(obj)
    Difficulty: Medium
    Answer: C. Object.keys(obj)
    Explanation: Object.keys() is a built-in method that returns an array of a given object's own enumerable string-keyed property names.
43. In an async function, how do you handle errors that might be thrown by an await expression?
    A. Using an if/else statement.
    B. Using a .catch() block chained inside the await.
    C. Wrapping the await expression in a try...catch block.
    D. Errors are automatically ignored in async functions.
    Difficulty: Medium
    Answer: C. Wrapping the await expression in a try...catch block.
    Explanation: Because async/await makes asynchronous code look synchronous, the standard synchronous try...catch block is the proper way to handle rejected promises.
44. What is the behavior of Promise.all() when passed an array of promises?
    A. It resolves as soon as the first promise resolves.
    B. It resolves only if all promises resolve, and rejects immediately if *any* single promise rejects.
    C. It ignores rejected promises and returns an array of only the successful ones.
    D. It executes the promises sequentially, one after the other.
    Difficulty: Medium
    Answer: B. It resolves only if all promises resolve, and rejects immediately if *any* single promise rejects.
    Explanation: Promise.all() is "all or nothing." It is used when you have multiple independent asynchronous tasks and you need all of them to finish successfully before moving on.
45. What is the difference between localStorage and sessionStorage in the browser?
    A. localStorage holds 5MB, sessionStorage holds 50MB.
    B. localStorage data persists even after the browser is closed; sessionStorage data is cleared when the page session ends (tab is closed).
    C. localStorage is encrypted, sessionStorage is not.
    D. localStorage can store objects natively, sessionStorage only stores strings.
    Difficulty: Medium

Answer: B. `localStorage` data persists even after the browser is closed; `sessionStorage` data is cleared when the page session ends (tab is closed).
Explanation: Both store data as strings natively, but `sessionStorage` is designed for temporary data tied to a specific browser tab's lifespan.

46. What is the purpose of the `Object.freeze()` method?
    A. It pauses JavaScript execution until the object is loaded.
    B. It deeply clones an object.
    C. It prevents new properties from being added to an object, and prevents existing properties from being removed or altered.
    D. It converts an object into a JSON string.
    Difficulty: Medium
    Answer: C. It prevents new properties from being added to an object, and prevents existing properties from being removed or altered.
    Explanation: `Object.freeze()` makes an object immutable. Note that it is shallow; if the frozen object contains another object, the inner object's properties can still be changed unless it is also frozen.
47. Which ES6 data structure is specifically designed to store a collection of strictly *unique* values?
    A. Map
    B. Array
    C. Set
    D. WeakMap
    Difficulty: Medium
    Answer: C. Set
    Explanation: A `Set` automatically removes duplicate entries. You can easily remove duplicates from an array by doing `[...new Set(myArray)]`.
48. What is the Nullish Coalescing Operator (`??`) used for?
    A. To strictly check for `null`.
    B. To return its right-hand operand when its left-hand operand is `null` or `undefined`, otherwise returning the left-hand operand.
    C. To replace the `&&` operator.
    D. To safely delete an object property.
    Difficulty: Medium
    Answer: B. To return its right-hand operand when its left-hand operand is `null` or `undefined`, otherwise returning the left-hand operand.
    Explanation: Unlike the logical OR operator (`||`) which triggers on *any* falsy value (like `0` or `""`), `??` only triggers on exactly `null` or `undefined`, making it safer for checking default values.
49. What does the Optional Chaining operator (`?.`) do?
    A. It checks if an API request is optional.
    B. It safely reads the value of a property located deep within a chain of connected objects without having to check that each reference in the chain is valid.
    C. It conditionally renders HTML in vanilla JS.
    D. It replaces ternary operators.

Difficulty: Medium
Answer: B. It safely reads the value of a property located deep within a chain of connected objects without having to check that each reference in the chain is valid.
Explanation: Instead of writing `if (user && user.address && user.address.street)`, you can write `user?.address?.street`. If any part is undefined, it short-circuits and returns `undefined` without throwing a TypeError.

50. Which Array method tests whether *at least one* element in the array passes the test implemented by the provided function?
    A. every()
    B. includes()
    C. some()
    D. any()
    Difficulty: Medium
    Answer: C. some()
    Explanation: `some()` returns a boolean. It stops iterating and returns `true` the moment it finds an element that satisfies the testing function. `every()` requires all elements to pass.
51. When a traditional function is used as an event listener callback (e.g., `button.addEventListener('click', myFunc)`), what does the `this` keyword inside `myFunc` refer to by default?
    A. The global `window` object.
    B. `undefined`.
    C. The element that fired the event (the button).
    D. The document body.
    Difficulty: Medium
    Answer: C. The element that fired the event (the button).
    Explanation: In standard event listeners, `this` is dynamically bound to the target element. (Note: If you use an arrow function instead, `this` will point to the surrounding lexical scope, not the button).
52. What is the difference between ES Modules (ESM) using `import`/`export` and CommonJS using `require()`?
    A. ESM is asynchronous and native to the browser, while CommonJS is synchronous and native to Node.js.
    B. CommonJS only works with classes.
    C. ESM cannot be used on servers.
    D. There is no difference; they are just different syntax for the exact same engine.
    Difficulty: Medium
    Answer: A. ESM is asynchronous and native to the browser, while CommonJS is synchronous and native to Node.js.
    Explanation: ES Modules are statically analyzed, allowing for tree-shaking, and load asynchronously. CommonJS loads modules synchronously, which works fine on a server's file system but blocks the browser.
53. What will `console.log(typeof NaN)` output?
    A. "NaN"
    B. "undefined"

C. "number"
D. "string"
Difficulty: Medium
Answer: C. "number"
Explanation: "Not-a-Number" is a special numeric value indicating an invalid number. Despite its name, its data type is strictly "number" in JavaScript.

54. If you assign an object to a new variable (`let obj2 = obj1;`) and change a property on `obj2`, what happens to `obj1`?
    A. `obj1` remains unchanged.
    B. `obj1` throws an error.
    C. The property is also changed on `obj1`.
    D. `obj2` becomes disconnected from `obj1`.
    Difficulty: Medium
    Answer: C. The property is also changed on `obj1`.
    Explanation: Objects (and arrays) in JavaScript are passed by reference, not by value. Both `obj1` and `obj2` point to the exact same location in memory.
55. How do you convert a NodeList (returned by `querySelectorAll`) into a true JavaScript Array so you can use methods like `map()` and `filter()`?
    A. NodeList.toArray()
    B. Array.from(nodeList) or [...nodeList]
    C. JSON.parse(nodeList)
    D. You cannot convert a NodeList.
    Difficulty: Medium
    Answer: B. Array.from(nodeList) or [...nodeList]
    Explanation: A NodeList is "array-like" but lacks most array methods. `Array.from()` or the spread operator easily converts it into a standard array.

Advanced Questions

56. In the context of the JavaScript Engine, what does the "JIT Compiler" do?
    A. It converts JavaScript into CSS.
    B. Just-In-Time compilation parses and compiles JavaScript code into native machine code on-the-fly during execution, rather than interpreting it line-by-line, to drastically improve performance.
    C. It compresses the bundle for the browser.
    D. It handles garbage collection.
    Difficulty: Advanced
    Answer: B. Just-In-Time compilation parses and compiles JavaScript code into native machine code on-the-fly during execution, rather than interpreting it line-by-line, to drastically improve performance.
    Explanation: Modern engines like Google's V8 use JIT. They monitor the code as it runs, identify "hot" functions that are called frequently, and compile them directly to machine code for maximum speed.

57. Why can manipulating the DOM directly inside a massive `for` loop cause the browser to freeze, and how do you fix it?
    A. The DOM is synchronous and recalculates layout on every change. Fix it by using a `DocumentFragment` to batch changes in memory before appending them to the DOM once.
    B. The DOM runs out of memory. Fix it by using `setTimeout`.
    C. Variables are hoisted poorly. Fix it with `let`.
    D. The DOM doesn't freeze; this is a myth.
    Difficulty: Advanced
    Answer: A. The DOM is synchronous and recalculates layout on every change. Fix it by using a `DocumentFragment` to batch changes in memory before appending them to the DOM once.
    Explanation: Appending elements one by one triggers the browser's expensive "Reflow" and "Repaint" processes repeatedly. A `DocumentFragment` exists in memory, allowing you to build the structure off-screen and trigger only one reflow.
58. What is the difference between deep copying an object with `JSON.parse(JSON.stringify(obj))` versus the modern `structuredClone(obj)`?
    A. `JSON` methods are much faster.
    B. `structuredClone` can safely copy cyclic references, Dates, Sets, and Maps, whereas the JSON method will crash or drop them.
    C. `structuredClone` only works on strings.
    D. There is no functional difference.
    Difficulty: Advanced
    Answer: B. `structuredClone` can safely copy cyclic references, Dates, Sets, and Maps, whereas the JSON method will crash or drop them.
    Explanation: `JSON.stringify` ignores functions, `undefined`, and Symbols, and it throws an error on circular references. `structuredClone` is a native browser API designed specifically for true deep copying.
59. What is a "Preflight Request" in CORS (Cross-Origin Resource Sharing)?
    A. A request sent to a CDN to warm up the cache.
    B. An automatic `OPTIONS` request sent by the browser before a complex HTTP request (like POST or PUT with custom headers) to verify if the server actually permits the cross-origin request.
    C. A request used to download fonts.
    D. A ping to check internet connection.
    Difficulty: Advanced
    Answer: B. An automatic `OPTIONS` request sent by the browser before a complex HTTP request (like POST or PUT with custom headers) to verify if the server actually permits the cross-origin request.
    Explanation: To protect servers from unexpected cross-origin actions, the browser securely "asks for permission" via an OPTIONS request before sending the actual data payload.
60. What is the fundamental difference between `Promise.race()` and `Promise.any()`?
    A. `race()` only works with two promises, `any()` works with arrays.

B. `race()` returns the first promise to settle (whether fulfilled or rejected), while `any()` returns the first promise to *fulfill*, ignoring early rejections unless all of them reject.
C. `any()` is a synchronous operation.
D. `race()` only returns rejections.
Difficulty: Advanced
Answer: B. `race()` returns the first promise to settle (whether fulfilled or rejected), while `any()` returns the first promise to *fulfill*, ignoring early rejections unless all of them reject.
Explanation: `race` is useful for creating timeouts (racing a fetch against a 5-second timer). `any` is useful when you request the same data from 3 different servers and just want the fastest successful response.

61. What is the purpose of the `Reflect` API in JavaScript?
A. To create mirrored DOM elements.
B. To provide methods for interceptable JavaScript operations, primarily used in tandem with the `Proxy` object to forward default behaviors safely.
C. To access private class variables.
D. To generate CSS reflections.
Difficulty: Advanced
Answer: B. To provide methods for interceptable JavaScript operations, primarily used in tandem with the `Proxy` object to forward default behaviors safely.
Explanation: When you create a Proxy trap (like overriding `set`), `Reflect.set()` allows you to perform the standard default assignment operation cleanly without writing manual object manipulation code.

62. What is a major limitation of running JavaScript inside a Web Worker?
A. It cannot use the `fetch` API.
B. It has absolutely no access to the DOM (`window` or `document` objects), meaning it cannot update the UI directly.
C. It can only execute for 5 seconds before timing out.
D. It only supports ES5 syntax.
Difficulty: Advanced
Answer: B. It has absolutely no access to the DOM (`window` or `document` objects), meaning it cannot update the UI directly.
Explanation: Web Workers run on a separate background thread to handle heavy CPU tasks. Because the DOM is not thread-safe, workers must use `postMessage` to send data back to the main thread, which then updates the UI.

63. Which JavaScript data type allows you to bypass the `Number.MAX_SAFE_INTEGER` limit (9,007,199,254,740,991) to perform math on astronomically large integers?
A. Double
B. Float64
C. BigInt
D. HugeNum
Difficulty: Advanced
Answer: C. BigInt
Explanation: Appending an `n` to the end of an integer literal (e.g., `10n`) creates a BigInt.

Note that you cannot mix BigInt and standard Number types in mathematical operations without explicitly casting them.

64. What does the `Object.defineProperty()` method allow you to do that a standard assignment (`obj.key = value`) does not?
    A. It allows you to use functions as values.
    B. It allows you to define intricate property descriptors, such as making a property read-only (`writable: false`), hidden from loops (`enumerable: false`), or using getter/setter functions.
    C. It allows you to bypass strict mode.
    D. It saves the object to the database.
    Difficulty: Advanced
    Answer: B. It allows you to define intricate property descriptors, such as making a property read-only (`writable: false`), hidden from loops (`enumerable: false`), or using getter/setter functions.
    Explanation: This method gives you fine-grained control over how a specific property behaves, which is essential when writing libraries or frameworks that require internal hidden properties.
65. What is the main characteristic of a `Symbol` in JavaScript?
    A. It is used to render SVG graphics.
    B. Every Symbol is guaranteed to be completely unique, making them ideal for creating hidden object properties that won't clash with other keys.
    C. It is a deprecated feature from ES3.
    D. It converts strings into memory addresses.
    Difficulty: Advanced
    Answer: B. Every Symbol is guaranteed to be completely unique, making them ideal for creating hidden object properties that won't clash with other keys.
    Explanation: Even if you create `Symbol('foo') === Symbol('foo')`, the result is `false`. Because they are unique, they don't show up in standard `for...in` loops or `Object.keys()`, effectively creating pseudo-private properties.
66. What is "Tail Call Optimization" (TCO) in recursive functions?
    A. Removing the last line of a function.
    B. An engine optimization where, if the last action of a function is a recursive call, the engine reuses the current stack frame instead of creating a new one, preventing Stack Overflow errors.
    C. Delaying the return statement.
    D. Executing functions in reverse order.
    Difficulty: Advanced
    Answer: B. An engine optimization where, if the last action of a function is a recursive call, the engine reuses the current stack frame instead of creating a new one, preventing Stack Overflow errors.
    Explanation: TCO allows recursive algorithms to run with constant memory space. Note that while in the ES6 spec, support among engines (like V8) is currently inconsistent or disabled.

67. What are `Atomics` and `SharedArrayBuffer` used for in JavaScript?
    A. Designing CSS animations.
    B. Allowing multiple Web Workers to safely read and write to the exact same block of memory simultaneously without locking the browser.
    C. Fetching data from GraphQL servers.
    D. Splitting strings into individual characters.
    Difficulty: Advanced
    Answer: B. Allowing multiple Web Workers to safely read and write to the exact same block of memory simultaneously without locking the browser.
    Explanation: Traditional Web Workers copy data back and forth. `SharedArrayBuffer` lets threads share memory, and `Atomics` provides synchronized operations to prevent race conditions when multiple threads edit that memory.
68. Why is the JavaScript engine described as "Single-Threaded, Non-Blocking, Asynchronous, and Concurrent"?
    A. Because it uses 4 separate processors.
    B. It has one Call Stack (single-threaded), but offloads tasks like network requests to Web APIs. The Event Loop then pushes the callbacks to the queue, allowing other code to run concurrently without blocking.
    C. It is a marketing term with no technical meaning.
    D. Because it only runs in Google Chrome.
    Difficulty: Advanced
    Answer: B. It has one Call Stack (single-threaded), but offloads tasks like network requests to Web APIs. The Event Loop then pushes the callbacks to the queue, allowing other code to run concurrently without blocking.
    Explanation: This architecture allows JavaScript to handle thousands of concurrent API requests on a Node.js server using only a single thread, whereas languages like Java or C# would spawn heavy individual threads for each request.
69. What happens if you try to strictly compare (`===`) two empty objects: `{} === {}`?
    A. It evaluates to `true`.
    B. It evaluates to `false`.
    C. It throws a SyntaxError.
    D. It returns `undefined`.
    Difficulty: Advanced
    Answer: B. It evaluates to `false`.
    Explanation: Objects are compared by their memory reference, not their structural content. Two distinct empty objects are created at two different locations in memory, so they are never strictly equal.
70. How does the `Event.preventDefault()` method differ from `Event.stopPropagation()`?
    A. They are identical.
    B. `stopPropagation()` stops the browser's default behavior (like submitting a form or following a link), while `preventDefault()` stops the event from bubbling up the DOM tree.
    C. `preventDefault()` stops the browser's default behavior (like a page reload on form submit), while `stopPropagation()` stops the event from bubbling up to parent elements.
    D. `preventDefault()` deletes the DOM element.

Difficulty: Advanced
Answer: C. `preventDefault()` stops the browser's default behavior (like a page reload on form submit), while `stopPropagation()` stops the event from bubbling up to parent elements.
Explanation: If you click a submit button, `preventDefault()` stops the page refresh. If that button is inside a clickable `div`, `stopPropagation()` ensures the div's click handler doesn't also fire.

Here is the final batch of your **JavaScript Mastery** quiz, completing the full set of 100 questions! This section covers **Questions 71 through 100**, focusing on DOM APIs, browser mechanics, modern data structures, and advanced performance APIs.

71. Which Array method is used to concatenate all elements of an array into a single string, separated by a specified separator string?
    A. concat()
    B. toString()
    C. join()
    D. merge()
    Difficulty: Easy
    Answer: C. join()
    Explanation: `join()` creates and returns a new string by concatenating all of the elements in an array. If you pass `"-"` as an argument, `[1, 2, 3]` becomes `"1-2-3"`.
72. What is the difference between the `DOMContentLoaded` event and the `window.onload` event?
    A. There is no difference.
    B. `window.onload` fires first, followed by `DOMContentLoaded`.
    C. `DOMContentLoaded` fires as soon as the HTML is fully parsed, while `window.onload` waits for all external resources (like images and stylesheets) to finish loading.
    D. `DOMContentLoaded` only fires on mobile devices.
    Difficulty: Easy
    Answer: C. `DOMContentLoaded` fires as soon as the HTML is fully parsed, while `window.onload` waits for all external resources (like images and stylesheets) to finish loading.
    Explanation: If you want to manipulate the DOM with JavaScript as quickly as possible without waiting for large images to load, `DOMContentLoaded` is the correct event to listen to.
73. What is JSON (JavaScript Object Notation)?
    A. A new programming language based on JavaScript.
    B. A lightweight data-interchange format that is easy for humans to read and write, completely language-independent but uses conventions familiar to C-family languages.
    C. A database built into the browser.
    D. A framework for building User Interfaces.
    Difficulty: Easy

Answer: B. A lightweight data-interchange format that is easy for humans to read and write, completely language-independent but uses conventions familiar to C-family languages.
Explanation: JSON is the universal standard for sending data across the web (like from a backend server to a frontend client), heavily inspired by JavaScript object syntax but usable in almost all programming languages.

74. How can you find the largest number in an array, such as `let nums = [5, 12, 8]`, using modern JavaScript?
    A. Math.max(nums)
    B. Math.largest(...nums)
    C. Math.max(...nums)
    D. nums.max()
    Difficulty: Easy
    Answer: C. Math.max(...nums)
    Explanation: `Math.max()` expects distinct variables as arguments (`Math.max(5, 12, 8)`). Using the spread operator (`...`) expands the array elements into individual arguments.
75. What does the `continue` statement do inside a loop?
    A. It stops the loop entirely.
    B. It skips the current iteration of the loop and immediately jumps to the next iteration.
    C. It pauses the loop for 1 second.
    D. It restarts the loop from the beginning.
    Difficulty: Easy
    Answer: B. It skips the current iteration of the loop and immediately jumps to the next iteration.
    Explanation: Unlike `break`, which destroys the loop entirely, `continue` just says "stop processing this specific item, but keep going with the rest of the list."
76. Which JavaScript keyword is used to access the parent class's constructor or methods from within a subclass?
    A. parent
    B. base
    C. super
    D. this
    Difficulty: Easy
    Answer: C. super
    Explanation: In ES6 classes, calling `super()` inside the `constructor` of a child class executes the constructor of the parent class, initializing the inherited properties.
77. What happens if you try to declare a variable with `let` twice in the exact same scope?
    A. The variable's value is updated to the second declaration.
    B. The first declaration is deleted.
    C. It throws a `SyntaxError`.
    D. It silently fails.
    Difficulty: Easy
    Answer: C. It throws a `SyntaxError`.
    Explanation: Unlike `var`, which allows you to accidentally redeclare the same variable

name multiple times, `let` and `const` throw an error, protecting you from accidentally overwriting your own variables.

78. Inside an event listener, what does `event.target` refer to?
    A. The element to which the event listener was explicitly attached.
    B. The specific DOM element that initiated the event (e.g., the exact span that was clicked inside a button).
    C. The URL of the page.
    D. The mouse coordinates.
    Difficulty: Easy
    Answer: B. The specific DOM element that initiated the event (e.g., the exact span that was clicked inside a button).
    Explanation: `event.target` is the actual element that was clicked. `event.currentTarget` refers to the element that the listener is attached to, which might be a parent element higher up in the tree.
79. True or False: Arrow functions have their own built-in `arguments` object containing all passed parameters.
    A. True, they work just like regular functions.
    B. False, arrow functions do not have an `arguments` object; they must use the rest parameter syntax (`...args`) instead.
    C. True, but only in strict mode.
    D. False, but neither do regular functions.
    Difficulty: Easy
    Answer: B. False, arrow functions do not have an `arguments` object; they must use the rest parameter syntax (`...args`) instead.
    Explanation: Traditional `function()` definitions automatically create an array-like `arguments` object. Arrow functions lack this, making the ES6 rest syntax the standard way to handle unknown numbers of arguments.
80. How do you convert a string like `"123.45"` into a true floating-point number in JavaScript?
    A. toFloat("123.45")
    B. parseFloat("123.45") or Number("123.45")
    C. Math.float("123.45")
    D. String.toNumber("123.45")
    Difficulty: Easy
    Answer: B. parseFloat("123.45") or Number("123.45")
    Explanation: `parseFloat` parses a string and returns a floating point number. `Number()` is also commonly used to explicitly coerce a string to a numeric type.

Medium Questions

81. What is the fundamental difference between a `Map` and a standard JavaScript `Object`?
    A. A Map is much slower than an Object.
    B. An Object can only have strings and Symbols as keys; a Map allows keys of any data type, including functions, arrays, and objects.
    C. A Map cannot be iterated over.

D. An Object remembers the insertion order of keys, while a Map does not.
Difficulty: Medium
Answer: B. An Object can only have strings and Symbols as keys; a Map allows keys of any data type, including functions, arrays, and objects.
Explanation: Maps are built specifically for data storage and key-value pairs, guaranteeing insertion order and allowing objects to be used as unique keys without converting them to strings.

82. What is the purpose of the `Object.create(null)` method?
A. To create an object that deletes itself.
B. To create a completely empty object that does not inherit from `Object.prototype`, meaning it has no default methods like `toString` or `hasOwnProperty`.
C. To create an object with `null` as its value.
D. To clear out an existing object.
Difficulty: Medium
Answer: B. To create a completely empty object that does not inherit from `Object.prototype`, meaning it has no default methods like `toString` or `hasOwnProperty`.
Explanation: Normal objects `{}` inherit prototype properties, which can cause key collisions if you use the object as a dictionary. `Object.create(null)` creates a "pure" dictionary.

83. What is the `MutationObserver` API used for in the browser?
A. To detect changes in CSS animations.
B. To watch for and react to changes being made to the DOM tree (like nodes being added, removed, or attributes modified).
C. To observe changes in the Redux store.
D. To track mouse movements globally.
Difficulty: Medium
Answer: B. To watch for and react to changes being made to the DOM tree (like nodes being added, removed, or attributes modified).
Explanation: `MutationObserver` replaced the old, slow DOM mutation events. It is heavily used by browser extensions to inject code the moment a specific element appears on a page.

84. How do `async` scripts and `defer` scripts differ when placed in the `<head>` of an HTML document?
A. `async` blocks HTML parsing; `defer` does not.
B. Both download in the background, but `async` executes as soon as it finishes downloading (potentially out of order), while `defer` waits until HTML parsing is complete and executes in the exact order they appear in the document.
C. `defer` only works for CSS.
D. `async` prevents the script from caching.
Difficulty: Medium
Answer: B. Both download in the background, but `async` executes as soon as it finishes downloading (potentially out of order), while `defer` waits until HTML parsing is complete and executes in the exact order they appear in the document.

Explanation: Use `async` for independent scripts like Google Analytics. Use `defer` for your main application scripts that depend on the DOM or other scripts being ready.

85. What is an `Iterable` in JavaScript?
    A. An object containing a specific loop method.
    B. Any object that implements the `Symbol.iterator` method, allowing it to be used in a `for...of` loop or with the spread operator.
    C. A deprecated array type.
    D. A variable that changes value over time.
    Difficulty: Medium
    Answer: B. Any object that implements the `Symbol.iterator` method, allowing it to be used in a `for...of` loop or with the spread operator.
    Explanation: Arrays, Strings, Maps, and Sets are all built-in iterables. You can make custom objects iterable by defining a generator function on their `[Symbol.iterator]` property.
86. What is the native JavaScript API used to format dates, numbers, and currencies according to the user's specific geographic locale?
    A. The `Intl` object (Internationalization API)
    B. The `Date.toLocaleString()` method exclusively
    C. The `Math.format()` method
    D. The `Moment.js` library
    Difficulty: Medium
    Answer: A. The `Intl` object (Internationalization API)
    Explanation: The `Intl` namespace provides language-sensitive string comparison, number formatting, and date/time formatting, removing the need for heavy third-party libraries for basic localization.
87. If you have a deeply nested array `[1, [2, [3, 4]]]`, which modern array method easily flattens it?
    A. reduce()
    B. concat()
    C. flat()
    D. merge()
    Difficulty: Medium
    Answer: C. flat()
    Explanation: The `flat()` method creates a new array with all sub-array elements concatenated into it recursively up to the specified depth (e.g., `array.flat(Infinity)`).
88. What does "Event Delegation" achieve in practical performance terms?
    A. It splits event listeners across multiple web workers.
    B. It reduces memory consumption by attaching a single event listener to a parent container instead of creating hundreds of individual listeners for child elements.
    C. It delegates JavaScript processing to the CSS engine.
    D. It prevents events from ever firing.
    Difficulty: Medium
    Answer: B. It reduces memory consumption by attaching a single event listener to a parent container instead of creating hundreds of individual listeners for child elements.

Explanation: Instead of looping over 500 list items to add a click listener to each, you add one listener to the <ul> and use event.target to figure out which <li> was clicked.

89. What is the difference between Array.prototype.slice() and Array.prototype.splice()?
    A. They do the exact same thing.
    B. slice() mutates the original array; splice() returns a new array without modifying the original.
    C. slice() returns a shallow copy of a portion of an array without modifying the original; splice() modifies the original array by removing, replacing, or adding elements.
    D. slice() only works on strings.
    Difficulty: Medium
    Answer: C. slice() returns a shallow copy of a portion of an array without modifying the original; splice() modifies the original array by removing, replacing, or adding elements.
    Explanation: Remember that slice is pure (no side effects), while splice is destructive (mutates the array in place).
90. In modern JavaScript, how do you mark a class property or method as strictly private?
    A. By prefixing it with an underscore _
    B. By using the private keyword
    C. By prefixing it with a hash #
    D. By wrapping it in a closure
    Difficulty: Medium
    Answer: C. By prefixing it with a hash #
    Explanation: While _ was a naming convention used by developers to imply privacy, modern ECMAScript standardizes the # prefix (e.g., #privateMethod()) to enforce strict privacy at the language level.

Advanced Questions

91. How do "Hidden Classes" (or "Inline Caching") improve performance in the V8 JavaScript Engine?
    A. They hide the source code from the user.
    B. V8 creates internal C++ structs (hidden classes) for objects sharing the exact same shape (keys and order). Accessing properties is optimized to a fast memory offset lookup rather than an expensive dictionary search.
    C. They encrypt the data automatically.
    D. They move the classes to a Web Worker.
    Difficulty: Advanced
    Answer: B. V8 creates internal C++ structs (hidden classes) for objects sharing the exact same shape (keys and order). Accessing properties is optimized to a fast memory offset lookup rather than an expensive dictionary search.
    Explanation: This is why dynamically adding or deleting properties from objects at random (delete obj.prop) destroys performance—it forces V8 to throw away the optimized hidden class and revert to slow dictionary mode.
92. Which API should be used to schedule background background tasks without impacting critical animations or the main thread's responsiveness?

A. requestAnimationFrame()
B. setTimeout(fn, 0)
C. requestIdleCallback()
D. Promise.resolve()
Difficulty: Advanced
Answer: C. requestIdleCallback()
Explanation: `requestIdleCallback` queues a function to be called during a browser's idle periods. It is perfect for low-priority work like sending analytics data without dropping 60fps animations.

93. In Node.js, what is the correct execution order of the Event Loop phases?
A. Poll -> Check -> Timers -> Close
B. Timers -> Pending Callbacks -> Idle/Prepare -> Poll -> Check -> Close Callbacks
C. Pending -> Timers -> Check -> Poll
D. Check -> Poll -> Timers -> Pending
Difficulty: Advanced
Answer: B. Timers -> Pending Callbacks -> Idle/Prepare -> Poll -> Check -> Close Callbacks
Explanation: Unlike the browser, Node.js has a highly specific multi-phase event loop handled by libuv. `setTimeout` fires in Timers, I/O callbacks in Poll, and `setImmediate` fires in the Check phase.

94. What is the purpose of `WeakRef` and `FinalizationRegistry` in JavaScript?
A. To create weak passwords.
B. To hold a weak reference to an object (allowing it to be garbage collected if no strong references exist) and register a callback to execute after it has been garbage collected.
C. To force the garbage collector to run immediately.
D. To permanently lock an object in memory.
Difficulty: Advanced
Answer: B. To hold a weak reference to an object (allowing it to be garbage collected if no strong references exist) and register a callback to execute after it has been garbage collected.
Explanation: These are advanced memory management tools. They allow you to build caches that don't cause memory leaks, allowing the engine to reclaim RAM when it needs to.

95. How does WebAssembly (Wasm) integrate with JavaScript?
A. Wasm replaces JavaScript entirely.
B. JavaScript loads the Wasm binary file, compiles it into a module, and can then call the highly-optimized Wasm functions directly from JS (and vice versa).
C. Wasm converts JavaScript to C++.
D. Wasm only runs on the backend.
Difficulty: Advanced
Answer: B. JavaScript loads the Wasm binary file, compiles it into a module, and can then call the highly-optimized Wasm functions directly from JS (and vice versa).
Explanation: Wasm is not a JS replacement; it's a complement. Heavy tasks (like video

editing or gaming engines) are written in Rust or C++, compiled to Wasm, and orchestrated by JavaScript.

96. What is the concept behind "Custom Elements" and the "Shadow DOM" (Web Components)?
    A. They are React specific features.
    B. They allow developers to create encapsulated, reusable HTML tags (`<my-element>`) where internal CSS styles and JavaScript do not leak out or clash with the rest of the document.
    C. They are used to hide elements from screen readers.
    D. They force rendering on the GPU.
    Difficulty: Advanced
    Answer: B. They allow developers to create encapsulated, reusable HTML tags (`<my-element>`) where internal CSS styles and JavaScript do not leak out or clash with the rest of the document.
    Explanation: Web Components are native browser APIs that offer component-based architecture without needing a framework like React or Vue, providing true style encapsulation via the Shadow DOM.
97. What does the `void` operator do in JavaScript, often seen as `void 0`?
    A. It deletes a variable.
    B. It evaluates the given expression and then returns `undefined`.
    C. It crashes the application.
    D. It creates an empty object.
    Difficulty: Advanced
    Answer: B. It evaluates the given expression and then returns `undefined`.
    Explanation: Because `undefined` is technically a global variable that could theoretically be overwritten in older browsers, minifiers and library authors use `void 0` as a mathematically bulletproof way to safely obtain the primitive `undefined` value.
98. Why do some developers use the double tilde operator (`~~`) instead of `Math.floor()` for positive numbers?
    A. It converts numbers to strings.
    B. It acts as a bitwise NOT operator twice. Because bitwise operations in JS implicitly convert floats to 32-bit integers, it acts as an extremely fast, albeit less readable, alternative to `Math.floor()`.
    C. It encrypts the number.
    D. It returns the absolute value.
    Difficulty: Advanced
    Answer: B. It acts as a bitwise NOT operator twice. Because bitwise operations in JS implicitly convert floats to 32-bit integers, it acts as an extremely fast, albeit less readable, alternative to `Math.floor()`.
    Explanation: `~~4.9` evaluates to `4`. While it is a micro-optimization, it is a common trick seen in high-performance graphic or gaming loops written in JavaScript.
99. What is an `ArrayBuffer` and how is it used with "Typed Arrays"?
    A. It buffers videos for streaming.
    B. It is a generic, fixed-length raw binary data buffer. You cannot manipulate it directly;

you must use a "Typed Array" view (like `Int32Array` or `Uint8Array`) to read and write the raw bytes.
C. It converts Arrays to Strings.
D. It stores Arrays in the cloud.
Difficulty: Advanced
Answer: B. It is a generic, fixed-length raw binary data buffer. You cannot manipulate it directly; you must use a "Typed Array" view (like `Int32Array` or `Uint8Array`) to read and write the raw bytes.
Explanation: Typed Arrays provide a mechanism to read and write raw binary data in memory. This is critical for WebGL, WebAssembly, and manipulating audio/video data directly in the browser.

100. When optimizing a vanilla JavaScript application, what is the most critical first step before altering any code?
A. Wrap everything in a Service Worker.
B. Move all code to Web Workers immediately.
C. Measure the performance using the browser's Performance tab (DevTools) to identify the actual bottlenecks, rather than guessing what might be slow.
D. Switch from `let` to `var`.
Difficulty: Advanced
Answer: C. Measure the performance using the browser's Performance tab (DevTools) to identify the actual bottlenecks, rather than guessing what might be slow.
Explanation: The "Golden Rule" of performance. Premature optimization introduces complex code. Using the DevTools profiler ensures you only spend time optimizing the functions that are actually blocking the main thread.